

Question Index - All 80 Questions

Q#	Question
Q1	What is Docker? How is it different from virtual machines?
Q2	Docker architecture - daemon, client, registry, images, containers
Q3	What is a Docker image vs a container?
Q4	Dockerfile basics - FROM, RUN, COPY, ADD, CMD, ENTRYPOINT
Q5	CMD vs ENTRYPOINT - when to use which
Q6	COPY vs ADD - why COPY is almost always the right choice
Q7	What are Docker layers? How does the layer cache work?
Q8	Multi-stage builds - why they exist and how they shrink images
Q9	Docker volumes - named, bind mounts, tmpfs
Q10	Docker networking - bridge, host, none, overlay
Q11	Expose vs publish (-p) - the difference that confuses everyone
Q12	.dockerignore - what it does and why it matters for build performance
Q13	Environment variables in Docker - ENV, ARG, -e, .env files
Q14	Docker Compose basics - services, networks, volumes, depends_on
Q15	docker exec vs docker attach - when to use each
Q16	Container lifecycle - create, start, stop, kill, rm, prune
Q17	Docker logs and debugging a crashing container
Q18	What is a Docker registry? Docker Hub vs ECR vs self-hosted
Q19	Tagging strategies - latest, semver, git SHA, date-based
Q20	Docker build context - what it is and why large contexts slow builds
Q21	Docker save vs export vs commit - what each does and when to use
Q22	Docker Desktop vs Docker Engine - differences and licensing

Q23	Dockerfile best practices - ordering, caching, minimal base images
Q24	Alpine vs Debian vs Distroless vs Scratch - choosing base images
Q25	Running containers as non-root - why and how
Q26	Health checks in Docker - HEALTHCHECK instruction and orchestrator integration
Q27	Docker Compose in production - profiles, override files, healthcheck
Q28	Container resource limits - memory, CPU, OOM killer behavior
Q29	Docker networking deep dive - DNS resolution, custom networks
Q30	Persistent storage patterns - database containers, shared volumes
Q31	Docker image scanning for vulnerabilities - Trivy, Snyk, Docker Scout
Q32	Slim images - reducing from 1GB to 50MB practical walkthrough
Q33	Docker build caching in CI/CD - BuildKit, registry cache, layer reuse
Q34	Multi-platform builds - buildx for amd64 and arm64
Q35	Docker secrets and sensitive data - build args vs secrets mount
Q36	Container logging strategies - stdout/stderr, log drivers, centralized logging
Q37	Docker init process - PID 1 problem, tini, dumb-init, zombie processes
Q38	Graceful shutdown - SIGTERM handling, shell vs exec form
Q39	Optimizing Docker for Python/Node/Java/Go - language-specific patterns
Q40	Docker-in-Docker vs Docker socket mounting - CI/CD tradeoffs
Q41	Private registries - ECR, GCR, Harbor, authentication
Q42	ARG scope, ONBUILD, and advanced Dockerfile instructions
Q43	Docker Compose v1 vs v2 - migration and behavioral differences
Q44	Container anti-patterns - fat containers, multiple processes
Q45	Docker daemon configuration - daemon.json, log rotation
Q46	Docker Swarm basics - when it still makes sense over Kubernetes

Q47	Container design patterns - sidecar, ambassador, adapter, init
Q48	Docker context and managing remote Docker hosts
Q49	Container internals - namespaces, cgroups, union filesystems
Q50	OCI standards - image spec, runtime spec, distribution spec
Q51	Docker vs containerd vs CRI-O vs Podman - runtime landscape
Q52	Rootless containers - setup, limitations, and security benefits
Q53	Docker storage drivers - overlay2, how layers mount
Q54	Container networking internals - veth pairs, iptables, bridge, DNS
Q55	Image layer deduplication, squashing, and registry garbage collection
Q56	BuildKit architecture - frontend, LLB, cache export/import
Q57	Docker in enterprise - governance, golden images, internal registries
Q58	Image signing and supply chain security - cosign, Notary, SBOM
Q59	Linux capabilities, seccomp profiles, AppArmor for containers
Q60	Docker Bench for Security - what it checks and how to act
Q61	Docker socket security - risks of mounting /var/run/docker.sock
Q62	Migrating from Docker Compose to Kubernetes
Q63	Container keeps restarting in a loop - debug step by step
Q64	Docker build taking 20 minutes - optimize to under 2 minutes
Q65	Image size is 2.3GB - reduce without breaking the application
Q66	Container runs locally but crashes in production
Q67	Secrets leaked in a Docker image layer - detect and remediate
Q68	Database container lost all data after restart
Q69	Two containers can't communicate on the same host
Q70	Docker Hub rate limits hit during CI

Q71	Production container compromised - incident response
Q72	Monolith to containerized microservices - migration strategy
Q73	Container OOM killed but no application logs
Q74	Volume permission denied - uid/gid mismatch
Q75	Zombie processes accumulating inside a container
Q76	/var/lib/docker eating 100% disk space
Q77	DNS resolution failing intermittently inside containers
Q78	App ignores SIGTERM and takes 30s to kill
Q79	Compromised base image in production - supply chain attack
Q80	Multi-stage build cache miss every CI run

SECTION 1 - Junior Core Concepts (Q1-Q22)

Q1 What is Docker? How is it different from virtual machines?

The Simple Answer

Docker is a platform that packages your application and all its dependencies into a lightweight, portable unit called a container. Unlike virtual machines that simulate an entire operating system with its own kernel, containers share the host OS kernel and isolate only the application layer. This makes containers start in seconds instead of minutes and use a fraction of the resources.

The Real-World Analogy

Think of shipping goods internationally. Before shipping containers were invented, every item was loaded individually onto ships - different shapes, different sizes, fragile items mixed with heavy ones. It was slow, expensive, and things broke constantly. Shipping containers standardized everything - pack your goods into a standard box and any ship, truck, or crane in the world can handle it. Docker containers did the same thing for software. Package your app into a standard container and it runs identically on any machine that has Docker installed.

VMs vs Containers - The Key Differences

A virtual machine includes a full guest operating system, its own kernel, virtualized hardware, and your application on top. Each VM might be 2-10 GB and take minutes to boot. A container shares the host kernel, has only the application and its dependencies, is typically 50-500 MB, and starts in under a second.

The critical difference is the isolation boundary. VMs provide hardware-level isolation - one VM crashing cannot affect another VM. Containers provide process-level isolation using Linux kernel features. This is lighter but the isolation is not as strong as VMs. For most applications this trade-off is worth it. For highly sensitive workloads requiring strict security boundaries, many teams still use VMs as the outer layer with containers inside.

When Containers Are the Wrong Choice

Containers are not always the answer. If your application requires a different kernel version, you need a VM. If you need to run Windows and Linux workloads on the same host, you need VMs. If compliance requires complete hardware isolation between tenants, containers alone may not satisfy auditors.

Follow-up Questions

- Can you run Docker containers inside a VM? Why would you?
- What is the performance overhead of containers vs bare metal?
- How does security isolation differ between containers and VMs?

Q2 Explain the Docker architecture - daemon, client, registry

The Simple Answer

Docker uses a client-server architecture. The Docker client is what you interact with when you type docker commands. The Docker daemon is the background service that actually builds images, runs containers, and manages everything. The registry is where images are stored and shared. These three components communicate over a REST API, usually over a Unix socket on the same machine.

How the Pieces Fit Together

When you type `docker run nginx`, the client sends that request to the daemon. The daemon checks if the `nginx` image exists locally. If not, it pulls it from the configured registry, usually Docker Hub. Then the daemon creates a container from that image and starts it. The client just displays the output - all the heavy lifting happens in the daemon.

The Analogy - A Restaurant Kitchen

The Docker client is the waiter taking your order. The Docker daemon is the kitchen doing the cooking. The registry is the wholesale supplier where the kitchen gets its ingredients. You tell the waiter what you want, the waiter tells the kitchen, the kitchen checks if it has the ingredients, orders from the supplier if needed, cooks the dish, and sends it back through the waiter.

Key Component Details

Docker Daemon (dockerd) - Listens for Docker API requests. Manages images, containers, networks, and volumes. Runs as a root process by default, which has security implications covered in Q61.

Docker Client (docker) - The primary way users interact with Docker. Can connect to local or remote daemons. When you configure Docker contexts, you can manage multiple Docker hosts from one client.

Docker Registry - A storage and distribution system for images. Docker Hub is the default public registry. Organizations typically run private registries like Amazon ECR, Google GCR, or self-hosted Harbor for proprietary images.

Q3 What is a Docker image vs a container?

The Simple Answer

An image is a read-only template containing everything needed to run an application - the code, runtime, libraries, and configuration. A container is a running instance of an image. You can create many containers from the same image, just like you can print many documents from the same template.

The Analogy - Class vs Object

If you come from a programming background, an image is like a class definition and a container is like an object instance. The class defines the blueprint. Each object created from it has its own state and lifecycle but shares the same blueprint. You can create a hundred objects from one class. Similarly, you can run a hundred containers from one image - each with its own filesystem layer, network identity, and process space.

What Happens Technically

When you run a container from an image, Docker creates a thin writable layer on top of the image's read-only layers. Any changes the container makes - new files, modified files, log output - go into this writable layer. When the container is deleted, this writable layer is deleted too. The image underneath remains unchanged and ready for the next container.

This is why containers are ephemeral by default. If you want data to survive container restarts or deletion, you must use volumes. This is one of the most common mistakes beginners make - storing important data inside the container filesystem and losing it when the container stops.

Q4 Dockerfile basics - FROM, RUN, COPY, ADD, CMD, ENTRYPOINT

The Simple Answer

A Dockerfile is a text file containing instructions to build a Docker image, step by step. Each instruction creates a new layer in the image. The most common instructions are FROM (base image), RUN (execute commands during build), COPY (add files from host), and CMD or ENTRYPOINT (what runs when the container starts).

The Core Instructions

FROM - Every Dockerfile starts with FROM. It sets the base image. FROM python:3.12-slim means start with a minimal Python 3.12 image. FROM scratch means start with nothing - used for statically compiled binaries.

RUN - Executes commands during the build process. Each RUN creates a new layer. Combine related commands with && to minimize layers: RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/* is better than three separate RUN commands.

COPY - Copies files from the build context into the image. COPY requirements.txt /app/ copies that file into the /app directory inside the image.

WORKDIR - Sets the working directory for subsequent instructions. Like running cd but persistent across layers.

CMD - The default command to run when a container starts. Can be overridden by passing a command to docker run.

ENTRYPOINT - The fixed command that always runs. Arguments passed to docker run get appended to ENTRYPOINT. See Q5 for the critical difference.

The Analogy - A Recipe

A Dockerfile is a recipe. FROM is choosing your base ingredients. RUN is the cooking steps. COPY is adding ingredients from your pantry. CMD is how you serve the dish. Just as a recipe produces the same dish every time if followed exactly, a Dockerfile produces the same image every time it is built.

Q5 CMD vs ENTRYPOINT - when to use which

The Simple Answer

ENTRYPOINT defines what your container IS - the fixed executable. CMD provides default arguments that can be overridden. If your container is a web server, ENTRYPOINT is the server binary. CMD might be the default configuration flags that users can override.

How They Interact

When both are specified, CMD becomes the default arguments to ENTRYPOINT. ENTRYPOINT sets the fixed base command and CMD provides overridable defaults. If a user passes arguments to docker run, those arguments replace CMD but ENTRYPOINT stays.

The Practical Rule

Use ENTRYPOINT when your container is designed to run a specific application - it should always run that application regardless of what arguments are passed. Use CMD when you want to provide a default command that the user might want to completely replace.

The common mistake: Using CMD for everything. When you use CMD alone, a user can accidentally replace your entire command by passing arguments to docker run. With ENTRYPOINT, the base command is locked in and users can only add arguments.

Shell Form vs Exec Form

Both CMD and ENTRYPOINT have shell form (CMD python app.py) and exec form (CMD ["python", "app.py"]). Always use exec form. Shell form wraps your command in /bin/sh -c, which means signals like SIGTERM go to the shell process instead of your application. This breaks graceful shutdown. See Q38 for why this matters in production.

Q6 COPY vs ADD - why COPY is almost always the right choice

The Simple Answer

COPY simply copies files from the build context into the image. ADD does the same thing but has two extra features - it can extract tar archives automatically and it can download files from URLs. Despite these extras, COPY is the recommended choice in nearly all cases because its behavior is predictable and transparent.

Why ADD Is Problematic

ADD's auto-extraction behavior is implicit and surprising. If you ADD a tar.gz file, it gets automatically extracted in the image. If you actually wanted the compressed file itself, ADD silently did the wrong thing. ADD's URL download feature is also discouraged because it creates large image layers that cannot be cleaned up in the same layer. Using RUN with curl or wget gives you more control - you can download, use, and delete a file in a single layer.

Docker's own best practices documentation says: Use COPY unless you specifically need ADD's tar extraction feature. This is one of those questions where knowing the recommendation and the reasoning behind it shows interviewer you follow best practices rather than just guessing.

Q7 What are Docker layers? How does the layer cache work?

The Simple Answer

Every instruction in a Dockerfile creates a new layer in the image. Layers are stacked on top of each other, and each layer only stores the differences from the layer below it. Docker caches these layers and reuses them when nothing has changed, which is what makes rebuilds fast.

The Analogy - Transparent Sheets

Think of overhead projector transparency sheets stacked on top of each other. Each sheet has some content on it. When you look through the whole stack from the top, you see the combined picture. If you change one sheet in the middle, you only need to replace that one sheet and everything above it - the sheets below remain unchanged. Docker layers work exactly the same way.

How Cache Invalidation Works

Docker checks each instruction against its cache. For RUN commands, it checks if the exact same command string was run before. For COPY and ADD, it checks the file checksums. If anything changes, that layer and every layer after it are rebuilt from scratch.

This is why instruction ordering in a Dockerfile matters enormously. Put things that change rarely at the top (base image, system packages) and things that change often at the bottom (application code). A change to your application code then only invalidates the last few layers instead of triggering a complete rebuild.

The golden rule: COPY requirements.txt before COPY application code. Install dependencies first. If only your code changes, the dependency installation layer is cached and reused. This can save minutes on every build.

Q8 Multi-stage builds - why they exist and how they shrink images

The Simple Answer

Multi-stage builds let you use multiple FROM statements in one Dockerfile. You compile or build your application in one stage with all the build tools, then copy only the final artifact into a clean minimal image in the next stage. The build tools and intermediate files are left behind, dramatically reducing the final image size.

The Analogy - Moving House

When you move to a new apartment, you pack everything into boxes at the old place. At the new place, you unpack only what you need and throw away the packing materials. Multi-stage builds work the same way - the first stage is the old apartment where you build everything. The second stage is the new apartment where you only bring the finished product.

Real-World Example

A Go application compiled in a full golang image with all build tools produces a single static binary. That binary can run in a scratch image with nothing else. The build stage might be 1.2 GB. The final image is 10 MB. The build tools, source code, and intermediate files are all discarded.

For Java applications, you might build with Maven in one stage and run with just the JRE in the next. For Node.js, you might install dependencies and build in one stage, then copy only the production `node_modules` and built files to the next.

When to Use Them

Use multi-stage builds for any compiled language or any application with a build step. Go, Java, C++, Rust, TypeScript, React - all benefit enormously. For interpreted languages like Python where the runtime IS the dependency, multi-stage builds help less but can still be used to exclude test dependencies, documentation, and build artifacts from the final image.

Q9 Docker volumes - named volumes, bind mounts, tmpfs

The Simple Answer

Volumes are Docker's mechanism for persisting data beyond the lifecycle of a container. When a container is deleted, its writable layer is gone. Volumes live outside the container filesystem and survive container restarts, stops, and even deletion.

Three Types of Volume Mounts

Named volumes - Managed entirely by Docker. Created with `docker volume create` or declared in `docker-compose`. Stored in `/var/lib/docker/volumes/` on the host. Best for databases and application data where you do not need direct host access.

Bind mounts - Map a specific host directory into the container. Changes on either side are immediately visible to the other. Best for development workflows where you edit code on your host and want the container to see changes instantly.

tmpfs mounts - Stored in the host memory only. Never written to disk. Best for sensitive data like credentials that should not persist and should not be written to the container's writable layer.

The Analogy - Storage Types

Named volumes are like a safety deposit box at the bank - Docker manages the location and access. Bind mounts are like a shared shelf between two rooms - both the host and container can see and modify the same files. Tmpfs is like a whiteboard - useful while you are working but erased when you are done.

The most common beginner disaster: Running a database container without a volume, filling it with data, and then losing everything when the container is recreated. Always declare volumes for stateful data.

Q10 Docker networking - bridge, host, none, overlay

The Simple Answer

Docker provides several network drivers that control how containers communicate with each other and with the outside world. The default is bridge networking, which creates an isolated network for containers on a single host.

Network Drivers Explained

Bridge - The default for standalone containers. Docker creates a virtual network switch. Containers on the same bridge can communicate using IP addresses. Containers on a custom bridge network can also communicate by container name via Docker's built-in DNS.

Host - The container shares the host's network stack directly. No network isolation. The container's ports are the host's ports. Fastest performance but no port mapping flexibility. Used when you need maximum network performance or when the application requires access to the host's network interfaces.

None - Complete network isolation. The container has no network access at all. Used for batch processing or security-sensitive workloads that should never communicate externally.

Overlay - Spans multiple Docker hosts. Used in Docker Swarm or when containers on different machines need to communicate as if they were on the same network. Creates an encrypted tunnel between hosts.

The Key Distinction That Matters

Always create custom bridge networks for your applications. The default bridge network does not support DNS-based container discovery - you have to use IP addresses which change when containers restart. Custom bridge networks resolve container names automatically, which is how Docker Compose works under the hood.

Q11 What is the difference between EXPOSE and publish (-p)?

The Simple Answer

EXPOSE is documentation in the Dockerfile - it declares which port the application inside the container listens on. It does not actually publish the port. The -p flag in docker run actually maps a container port to a host port, making it accessible from outside. EXPOSE without -p does nothing functional. -p without EXPOSE works perfectly fine.

Think of EXPOSE as a label on a building saying 'entrance here.' It tells people where the door is but does not open it. The -p flag actually opens the door and connects it to the street.

Q12 What is .dockerignore and why does it matter for build performance?

The Simple Answer

The .dockerignore file tells Docker which files and directories to exclude from the build context. When you run docker build, Docker sends the entire build context directory to the daemon. If that directory contains node_modules, .git, large data files, or local environment files, you are sending gigabytes of unnecessary data that slows down every build.

What to Always Include in .dockerignore

.git, node_modules, __pycache__, .env files, local IDE configs, test data, documentation, and any build artifacts that will be regenerated inside the container. A good .dockerignore can reduce build context from gigabytes to megabytes and cut build startup time from minutes to seconds.

Q13 Environment variables in Docker - ENV, ARG, -e, .env files

The Simple Answer

There are multiple ways to pass configuration to Docker containers. ENV sets variables in the image that persist into running containers. ARG sets variables available only during build time. The -e flag passes variables at runtime. The .env file provides defaults for Docker Compose.

The Critical Differences

ARG - Available only during docker build. Not visible in the running container. Used for build-time configuration like version numbers, base image tags, or build flags. Can be set with --build-arg.

ENV - Baked into the image and available in the running container. Used for application configuration like database URLs, API keys (though secrets are better for sensitive values), or feature flags.

-e flag - Passed at docker run time. Overrides ENV values from the image. Best for environment-specific configuration that changes between deployments.

Security warning: ARG values are visible in the image history via docker history. Never pass secrets through ARG. Use Docker BuildKit's secret mount feature instead. See Q35 for details.

Q14 Docker Compose basics - services, networks, volumes, depends_on

The Simple Answer

Docker Compose is a tool for defining and running multi-container applications using a YAML file. Instead of running multiple docker run commands with long argument lists, you declare all your services, their configurations, networks, and volumes in one docker-compose.yml file and bring everything up with docker compose up.

Core Concepts

Services - Each service is a container definition. A typical web application might have three services: the application, a database, and a reverse proxy.

Networks - Compose creates a default network for all services. Services can reach each other by service name. You can define additional networks to isolate groups of services.

Volumes - Declared at the top level and referenced by services. Named volumes persist data across container restarts.

depends_on - Declares startup order. But critically, depends_on only waits for the container to start, not for the application inside to be ready. A database container might start in one second but take ten seconds to accept connections. Use healthchecks for proper readiness waiting.

Q15 docker exec vs docker attach - when to use each

The Simple Answer

docker exec runs a new command inside a running container - like opening a new terminal window into the container. docker attach connects your terminal to the container's main running process - its PID 1. The difference matters because attaching to PID 1 means Ctrl+C sends SIGINT to the main process, which can stop your container. Exec starts a separate process, so exiting it leaves the container running.

Practical rule: Use `docker exec -it container_name /bin/sh` for debugging. Use `docker attach` only when you specifically need to interact with the main process. Most people should never use `attach` in production.

Q16 Container lifecycle - create, start, stop, kill, rm, prune

The Simple Answer

Containers go through a predictable lifecycle. `docker create` sets up the container without starting it. `docker start` begins execution. `docker stop` sends `SIGTERM` and waits for graceful shutdown, then sends `SIGKILL` after a timeout. `docker kill` sends `SIGKILL` immediately. `docker rm` removes a stopped container. `docker system prune` cleans up all unused resources.

The Important Nuance

`docker stop` gives the process time to clean up - close database connections, finish in-flight requests, save state. `docker kill` terminates immediately with no cleanup. Always prefer `stop` in production. `Kill` is for containers that are hung and unresponsive to `SIGTERM`.

The cleanup commands every engineer should know: `docker system prune` removes stopped containers, unused networks, dangling images, and build cache. `docker system prune -a` also removes unused images. `docker volume prune` removes orphaned volumes. Run these periodically on CI runners and development machines to reclaim disk space.

Q17 Docker logs and debugging a crashing container

The Simple Answer

`docker logs` shows the `stdout` and `stderr` output of a container. For a crashing container, `docker logs container_name` shows what the application printed before it died. The `--tail` flag shows the last N lines, and `-f` follows the log output in real time.

Debugging Steps for a Crashing Container

- Start with `docker logs` to see error output
- Check `docker inspect` for the exit code - 137 means OOM killed, 1 means application error, 139 means segfault
- If the container crashes too fast to `exec` into, override the `entrypoint`: `docker run -it --entrypoint /bin/sh image_name` to get a shell and investigate manually
- Check `docker events` for system-level issues
- Use `docker stats` to see if the container is hitting resource limits

The exit code is your first clue. Exit code 0 means clean exit. Non-zero means error. Code 137 (128+9) means the container was killed with `SIGKILL`, often by the OOM killer. Code 143 (128+15) means `SIGTERM` was received and the process exited. Knowing these codes speeds up debugging enormously.

Q18 What is a Docker registry? Docker Hub vs ECR vs self-hosted

The Simple Answer

A Docker registry is a storage and distribution service for Docker images. It is where you push images after building them and where Docker pulls images from when running containers. Docker Hub is the default public registry. Most organizations use private registries for proprietary images.

Comparison

Docker Hub - Free for public images. Rate-limited for pulls (100 pulls per 6 hours for anonymous users, 200 for authenticated). Has official images maintained by Docker and verified publisher images.

Amazon ECR - Fully managed private registry integrated with AWS IAM. No pull rate limits within AWS. Supports image scanning, replication across regions, and lifecycle policies to clean up old images.

Google GCR / Artifact Registry - Google Cloud's equivalent. Integrated with GCP IAM and Cloud Build.

Harbor - Open-source self-hosted registry by the CNCF. Provides image scanning, signing, replication, and role-based access. Good for organizations that want full control or cannot use cloud registries.

Q19 Tagging strategies - latest, semver, git SHA, date-based

The Simple Answer

Tags are labels attached to Docker images that identify specific versions. A well-designed tagging strategy makes deployments traceable, rollbacks reliable, and debugging faster.

Common Strategies

latest - The default tag. Problematic because it is mutable - it points to whatever was pushed last. Two machines pulling latest at different times may get different images. Never use latest in production. Never.

Semantic versioning (semver) - Tags like v1.2.3. Clear, human-readable, supports rollback to specific versions. Best for released software.

Git SHA - Tags like abc123f. Every image is traceable to an exact commit. Best for CI/CD pipelines where you want to know exactly what code is running. Cannot easily tell which is newer without checking git history.

Date-based - Tags like 2025-06-15. Good for nightly builds or snapshot releases.

The recommended practice: Tag every image with both the git SHA and a semver tag when releasing. Tag CI builds with just the SHA. Never rely on latest for anything except local development.

Q20 Docker build context - what it is and why large contexts slow builds

The Simple Answer

The build context is the set of files and directories that Docker sends to the daemon when you run docker build. Whatever directory you specify as the build path - usually the current directory with the dot (.) - gets packaged and sent. If that directory is 2 GB, Docker sends 2 GB to the daemon before even starting the build.

Why This Matters

The build context transfer happens on every build. If you are building from a project root that contains node_modules (500 MB), a .git directory (200 MB), test fixtures (100 MB), and data files (500 MB), you are sending 1.3 GB on every single build even if your Dockerfile only COPYs a few small files. Use .dockerignore to exclude everything the Dockerfile does not need.

Q21 Docker save vs export vs commit - what each does and when to use

The Simple Answer

These three commands move images and containers around in different ways and people constantly confuse them.

docker save - Saves an image (with all its layers and tags) to a tar archive. Used to transfer images between machines that do not share a registry. `docker save myimage > image.tar` then `docker load < image.tar` on the other machine.

docker export - Saves a container's filesystem to a tar archive. Flattens all layers into one. Loses Dockerfile metadata like CMD, ENTRYPOINT, ENV. Used for inspecting container filesystem contents.

docker commit - Creates a new image from a running container's changes. The container's writable layer becomes a new image layer. Tempting for quick fixes but terrible for reproducibility - the resulting image has no Dockerfile to rebuild from.

Production rule: Never use docker commit for anything that goes to production. If you cannot rebuild an image from a Dockerfile, you have an unmanageable snowflake. Commit is acceptable only for debugging - taking a snapshot of a broken container to analyze later.

Q22 Docker Desktop vs Docker Engine - differences and licensing

The Simple Answer

Docker Engine is the free, open-source container runtime that runs on Linux. Docker Desktop is a commercial product that packages Docker Engine inside a lightweight VM for macOS and Windows, adding a GUI, Kubernetes integration, volume sharing, and automatic updates.

The Licensing Question Interviewers Ask

Docker Desktop requires a paid subscription for companies with more than 250 employees or more than \$10 million in annual revenue. Docker Engine remains free and open-source. Many organizations switched their developer environments to alternatives like Podman Desktop, Rancher Desktop, or Colima for macOS to avoid the licensing cost.

Know this for interviews: The licensing distinction between Desktop and Engine is a surprisingly common question, especially at cost-conscious organizations. Being able to articulate the alternatives shows awareness of the broader container ecosystem.

SECTION 2 - Mid-Level Production Patterns (Q23-Q48)

Q23 Dockerfile best practices - ordering, caching, minimal base images

The Simple Answer

A well-written Dockerfile builds fast, produces small images, and is secure by default. The three pillars are: order instructions for maximum cache reuse, combine commands to minimize layers, and start from the smallest base image that works.

Instruction Ordering for Cache

Put instructions that change least frequently at the top. Base image and system packages change rarely - put them first. Application dependencies change occasionally - put them in the middle. Application source code changes on every commit - put it last. This way, a code change only rebuilds the last few layers.

Layer Minimization

Every RUN creates a layer. Combine related commands: `RUN apt-get update && apt-get install -y package && rm -rf /var/lib/apt/lists/*` instead of three separate RUN lines. This keeps layers small and avoids caching stale package lists.

Security Defaults

Always run as a non-root user. Always use specific version tags, never latest. Always scan the final image for vulnerabilities before pushing. Set HEALTHCHECK so orchestrators can detect unhealthy containers.

The Dockerfile that passes a senior code review: Uses a specific minimal base image, installs dependencies before copying code, runs as non-root, has a HEALTHCHECK, uses multi-stage builds, has no secrets in any layer, and produces an image under 200 MB.

Q24 Alpine vs Debian vs Distroless vs Scratch - choosing base images

The Simple Answer

Your base image choice affects image size, security surface, debugging capability, and compatibility. There is no universally best choice - it depends on your priorities.

Alpine - 5 MB base. Uses musl libc instead of glibc. Very small but can cause subtle compatibility issues with some Python packages, DNS resolution, and performance in certain workloads. Great for Go and statically compiled applications.

Debian slim - 80 MB base. Uses glibc. Maximum compatibility with minimal size. The default recommendation for most production workloads. Predictable behavior with no surprises.

Distroless - Google's images containing only the application runtime with no shell, no package manager, no operating system utilities. Excellent security posture but difficult to debug because you cannot exec into the container.

Scratch - Literally empty. Zero bytes. Only works for statically compiled binaries. The smallest possible image.

The decision framework: Start with Debian slim. Switch to Alpine only if you have tested thoroughly and the size reduction matters. Use Distroless for hardened production images where security outweighs debuggability. Use scratch for Go or Rust binaries.

Q25 Running containers as non-root - why and how

The Simple Answer

By default, processes inside Docker containers run as root. If an attacker exploits a vulnerability in your application, they have root access inside the container. While container isolation limits what root can do, running as non-root adds defense in depth. Container escape vulnerabilities combined with root access inside the container give the attacker root on the host.

How to Implement

Add a USER instruction in your Dockerfile after installing packages and before the CMD. Create a dedicated user and group: `RUN groupadd -r appuser && useradd -r -g appuser appuser`, then `USER appuser`. Ensure your application files are owned by this user and writable directories exist.

The gotcha that trips people up: If your application writes to files or directories, those must be writable by the non-root user. Create and `chown` directories before switching to the non-root user. A common pattern is `RUN mkdir -p /app/data && chown -R appuser:appuser /app` before the `USER` instruction.

Q26 Health checks in Docker - HEALTHCHECK instruction

The Simple Answer

HEALTHCHECK tells Docker how to test whether a container is still working correctly. Without a health check, Docker only knows if the main process is running - it has no idea if the application is actually responding to requests. A container can be running but completely broken (deadlocked, out of memory, unable to reach a database).

How It Works

HEALTHCHECK defines a command that Docker runs periodically inside the container. If the command exits with 0, the container is healthy. If it exits with 1, the container is unhealthy. Docker tracks the health status and orchestrators like Swarm or Kubernetes use it to decide whether to restart or replace the container.

Common Health Check Patterns

- HTTP endpoint: `HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1`
- Database: `HEALTHCHECK CMD pg_isready -U postgres || exit 1`
- Custom script: `HEALTHCHECK CMD /app/healthcheck.sh`

Do not use curl for health checks in production. It adds curl as a dependency. Use `wget -q --spider` instead (available in Alpine), or write a small binary health check in your application language that checks the actual functionality.

Q27 Docker Compose in production - profiles, override files, healthcheck

The Simple Answer

Docker Compose started as a development tool but is increasingly used in production for single-host deployments. Profiles let you selectively enable services. Override files let you layer environment-specific configuration. Healthchecks with `depends_on` conditions provide proper startup ordering.

Profiles

Profiles let you group services so only some start by default. A debug service or a monitoring stack can be in a profile that is only activated when needed, keeping the default deployment lean.

Override Files

docker-compose.yml contains base configuration. docker-compose.override.yml contains development-specific settings and is loaded automatically. For production, use docker compose -f docker-compose.yml -f docker-compose.prod.yml to layer production settings without touching the base file.

Proper Startup Ordering

depends_on with condition: service_healthy waits for the health check to pass before starting dependent services. This solves the common problem of an application starting before its database is ready.

Q28 Container resource limits - memory, CPU, OOM killer behavior

The Simple Answer

Without resource limits, a container can consume all available host memory and CPU, starving other containers and the host itself. Docker lets you set hard limits on memory and CPU using cgroups.

Memory Limits

Set with --memory or mem_limit in Compose. When a container exceeds its memory limit, the Linux OOM killer terminates it. The container exits with code 137. There is no warning and no graceful shutdown - the process is killed immediately.

CPU Limits

--cpus limits the total CPU time. --cpu-shares sets relative priority when competing for CPU. --cpuset-cpus pins a container to specific CPU cores. For most applications, --cpus is sufficient.

Always set memory limits in production. A memory leak in one container without limits can crash the entire host, taking down every other container with it. Setting limits contains the blast radius to a single container.

Q29 Docker networking deep dive - DNS resolution, custom networks

The Simple Answer

Docker's built-in DNS server allows containers on the same custom network to find each other by name. This eliminates hardcoded IP addresses and is the foundation of service discovery in Docker.

How DNS Resolution Works

When you create a custom bridge network and attach containers to it, Docker runs an embedded DNS server at 127.0.0.11 inside each container. When container A tries to connect to container B by name, the embedded DNS resolves the name to container B's IP on that network. If the container is recreated with a new IP, DNS automatically updates.

The Default Bridge Network Gotcha

The default bridge network does not support DNS resolution. Containers on the default bridge must use IP addresses or legacy --link flags. This is why Docker Compose creates a custom network for every project - so service-to-service discovery works by name automatically.

Q30 Persistent storage patterns - database containers, shared volumes

The Simple Answer

Running databases in containers is common for development and increasingly for production, but it requires understanding persistent storage patterns to avoid data loss.

Named Volumes for Databases

Always mount the database data directory to a named volume. For PostgreSQL, mount `/var/lib/postgresql/data`. For MySQL, mount `/var/lib/mysql`. This ensures data survives container recreation. Back up volumes regularly with `docker run --rm -v volume_name:/data -v $(pwd):/backup alpine tar czf /backup/backup.tar.gz /data`.

Volume Drivers for Production

The default local volume driver stores data on the host filesystem. For production, consider volume drivers that provide replication, snapshots, and backup - like REX-Ray for cloud block storage or NFS drivers for shared storage.

Never run a production database in a container without a documented backup and recovery strategy. Containers make it easy to spin up a database and dangerously easy to lose all its data.

Q31 Docker image scanning for vulnerabilities - Trivy, Snyk, Docker Scout

The Simple Answer

Image scanning checks every package and library in your Docker image against known vulnerability databases (CVEs). It catches security issues before they reach production.

The Major Tools

Trivy - Open-source, fast, runs locally or in CI. Scans OS packages and application dependencies. The most popular open-source option.

Docker Scout - Built into Docker Desktop and Docker Hub. Provides continuous monitoring and remediation advice.

Snyk - Commercial with free tier. Strong developer experience with fix suggestions and PR integration.

Practical Integration

Run scans in CI on every build. Block deployments if critical or high vulnerabilities are found. Accept medium and low vulnerabilities with a review process. Update base images regularly - most vulnerabilities come from the base image, not your code.

Scanning alone is not enough. You need a process for acting on findings. A scan that reports 200 vulnerabilities every day that nobody looks at is security theater. Start with blocking critical issues and expand coverage gradually.

Q32 Slim images - reducing from 1GB to 50MB practical walkthrough

The Simple Answer

Large images waste bandwidth, slow deployments, increase attack surface, and cost more in registry storage. Most images can be reduced by 80-95% with systematic optimization.

The Reduction Steps

- Switch from a full OS base (ubuntu:22.04 at 78 MB) to a slim variant (python:3.12-slim at 45 MB)
 - Use multi-stage builds to exclude build tools from the final image
 - Combine RUN instructions and clean up package manager caches in the same layer
 - Remove unnecessary files - docs, man pages, test files, build artifacts
 - For compiled languages, use scratch or distroless as the final stage
 - Use `.dockerignore` to prevent large files from entering the build context
-

Q33 Docker build caching in CI/CD - BuildKit, registry cache, layer reuse

The Simple Answer

CI/CD environments lose the local Docker cache between runs because each build gets a fresh VM or container. This means every CI build starts from scratch unless you explicitly configure remote caching.

BuildKit Cache Solutions

BuildKit supports exporting and importing cache to and from a registry. You push cache metadata alongside your image, and subsequent builds pull the cache before building. This brings CI build speeds close to local development speeds.

Registry Cache Pattern

Build with `--cache-to` and `--cache-from` flags. The first build pushes cache layers to the registry. Subsequent builds pull cache layers before building, reusing unchanged layers. Combined with good Dockerfile layer ordering, this can reduce CI build times from 15 minutes to 2 minutes.

Q34 Multi-platform builds - buildx for amd64 and arm64

The Simple Answer

Docker buildx extends the build command to create images for multiple CPU architectures from a single Dockerfile. This is essential now that ARM-based processors are mainstream - AWS Graviton instances, Apple Silicon Macs, and Raspberry Pi all use ARM.

How It Works

buildx uses QEMU emulation or native builder nodes to compile for each target architecture. You build once and push a multi-platform manifest to the registry. When someone pulls the image, Docker automatically selects the correct architecture variant.

The practical need: A developer on an M1/M2 Mac builds an image. That image needs to run on AMD64 Linux servers in production. Without multi-platform builds, the Mac-built image fails on the server. With buildx, one command produces images for both architectures.

Q35 Docker secrets and sensitive data - build args vs secrets mount

The Simple Answer

Handling secrets in Docker builds is tricky because anything passed through ARG or ENV can be extracted from image layers. BuildKit's secret mount feature solves this by mounting secrets during build without persisting them in any layer.

What Leaks and What Doesn't

ARG values - Visible in docker history. Never use for passwords or API keys.

ENV values - Baked into the image. Visible to anyone who pulls the image.

COPY of a secret file - Persists in the image layer even if you delete it in a later RUN step. Layers are additive - previous layers still contain the file.

BuildKit secret mounts - Mounted temporarily during a single RUN step. Never written to any layer. This is the correct approach for build-time secrets like private registry credentials or API keys needed during compilation.

The nightmare scenario: An engineer COPYs a .env file containing database passwords into the image, uses them, then deletes the file in the next RUN. The passwords are permanently in the COPY layer. Anyone who pulls the image can extract them with `docker save` and `tar`.

Q36 Container logging strategies - stdout/stderr, log drivers, centralized logging

The Simple Answer

Docker captures everything written to stdout and stderr by the container's main process. This is the twelve-factor app principle - applications should log to stdout and let the infrastructure handle routing, storage, and analysis.

Log Drivers

Docker supports multiple log drivers that control where logs go. The json-file driver (default) writes to disk. The syslog driver sends to syslog. The fluentd and gelf drivers send to centralized logging systems. The awslogs driver sends directly to CloudWatch.

Centralized Logging Pattern

Run a log aggregator like Fluentd or Fluent Bit as a container on each host. Configure Docker to use the fluentd log driver. The aggregator collects, transforms, and forwards logs to Elasticsearch, Loki, or CloudWatch. Applications just write to stdout - they never need to know about the logging infrastructure.

Set log rotation on the json-file driver. Without rotation, container logs grow unbounded and will eventually fill the disk. Configure `max-size` and `max-file` in `daemon.json` or per-container.

Q37 Docker init process - PID 1 problem, tini, dumb-init, zombie processes

The Simple Answer

In Linux, PID 1 has special responsibilities - it must reap zombie processes and forward signals to child processes. Most applications are not designed to be PID 1. When your application runs as PID 1 inside a container, zombie processes accumulate and signals like SIGTERM may not reach your application properly.

The Problem

Your web server starts as PID 1 in the container. It spawns worker processes. When a worker crashes, it becomes a zombie process waiting for PID 1 to reap it. Your web server does not know it should reap zombies. Over time, zombies accumulate, consuming process table entries until the container cannot start new processes.

The Solution - Init Process

Use an init process like tini or dumb-init as PID 1. It handles signal forwarding and zombie reaping. Docker has a built-in option: `docker run --init` which uses tini. In a Dockerfile, you can install tini and set it as the ENTRYPOINT.

Docker's --init flag or the tini binary should be the default for any long-running container. It costs nothing in performance and prevents an entire class of production issues. Many engineers only discover the PID 1 problem after a container has been running for weeks and starts behaving erratically.

Q38 Graceful shutdown - SIGTERM handling, shell vs exec form

The Simple Answer

When Docker stops a container, it sends SIGTERM to PID 1 and waits for a grace period (default 10 seconds). If the process does not exit, Docker sends SIGKILL which terminates it immediately. The difference between a graceful and ungraceful shutdown is whether your application receives and handles SIGTERM.

Shell Form vs Exec Form - The Trap

CMD python app.py (shell form) wraps your command in /bin/sh -c. The shell process becomes PID 1 and receives SIGTERM. Most shells do not forward signals to child processes. Your Python application never receives SIGTERM. After 10 seconds, Docker sends SIGKILL, killing everything without cleanup.

CMD ["python", "app.py"] (exec form) makes your Python application PID 1 directly. It receives SIGTERM and can handle it - closing connections, flushing buffers, finishing in-flight requests.

This is one of the most common causes of slow container shutdowns and data loss in production.

Always use execform. Always handle SIGTERM in your application. Always test that your container stops cleanly within the grace period.

Q39 Optimizing Docker for Python/Node/Java/Go - language-specific patterns

Python

COPY requirements.txt first, RUN pip install, then COPY application code. This caches the expensive dependency installation. Use --no-cache-dir with pip to avoid storing downloaded packages. Use slim or distroless Python images. Consider compiling to .pyc files and excluding .py source.

Node.js

COPY package.json and package-lock.json first, RUN npm ci (not npm install - ci is deterministic). Then COPY source code. Use node:slim or node:alpine. Set NODE_ENV=production to exclude devDependencies. Consider multi-stage: build with full image, run with slim image containing only production dependencies.

Java

Multi-stage is essential. Build with Maven or Gradle in one stage. Copy only the JAR to a JRE-only image in the second stage. Use eclipse-temurin:21-jre-alpine for a minimal runtime. Consider jlink to create a custom JRE containing only the modules your application needs.

Go

The easiest language for Docker optimization. Build in golang image, copy the static binary to scratch. Final image can be under 10 MB. Use CGO_ENABLED=0 for fully static binaries. Multi-platform builds work effortlessly with Go's cross-compilation support.

Q40 Docker-in-Docker vs Docker socket mounting - CI/CD tradeoffs

The Simple Answer

CI pipelines need to build Docker images. There are two approaches: running a Docker daemon inside a Docker container (DinD) or mounting the host's Docker socket into the CI container.

Docker-in-Docker (DinD)

Runs a complete Docker daemon inside a privileged container. Each CI job gets an isolated Docker environment. Requires --privileged which gives the container almost full host access - a security concern.

Socket Mounting

Mounts /var/run/docker.sock from the host into the CI container. The CI container uses the host's Docker daemon. Simpler and faster but less isolated - all CI jobs share the host daemon, can see each other's containers and images, and can potentially affect the host. See Q61 for security implications.

The modern recommendation: Use BuildKit's standalone mode or Kaniko for building images in CI without needing a Docker daemon at all. Both work in unprivileged containers and are designed for CI environments. DinD and socket mounting are legacy patterns that remain common but carry security risks.

Q41 Private registries - ECR, GCR, Harbor, authentication

The Simple Answer

Private registries store proprietary images securely. Authentication prevents unauthorized access. Pull-through caches reduce bandwidth and dependency on external registries.

ECR Workflow

ECR tokens expire every 12 hours. Use the AWS CLI to get tokens automatically in CI. Configure lifecycle policies to clean up old images - without them, storage costs grow indefinitely. Enable image scanning on push.

Harbor for Self-Hosted

Harbor provides vulnerability scanning, image signing, replication across sites, RBAC, and audit logging. Deploy it when you need full control over your registry or when compliance requires on-premises image storage.

Q42 ARG scope, ONBUILD, and advanced Dockerfile instructions

ARG Scope Across Stages

ARG values defined before FROM are only available in the FROM instruction itself. ARG values defined after FROM are available in that stage only. To use an ARG in multiple stages, redeclare it in each stage. This scoping rule catches people who expect build args to be global.

ONBUILD

ONBUILD registers instructions that execute when the image is used as a base image in another Dockerfile. Used for creating base images that automatically trigger build steps in child images. Rarely used in practice because it creates hidden magic that is difficult to debug.

Q43 Docker Compose v1 vs v2 - migration and behavioral differences

The Simple Answer

Docker Compose v1 was a standalone Python application (docker-compose). Compose v2 is a Go plugin integrated into the Docker CLI (docker compose - space instead of hyphen). v1 reached end-of-life in July 2023. v2 is faster, uses Docker's API directly, and supports newer Compose specification features.

Key Behavioral Differences

Container naming changed - v2 uses hyphens instead of underscores by default. Build behavior differs slightly in cache handling. Some v1 flags were renamed or removed. Environment variable interpolation rules changed slightly.

If your CI/CD scripts use docker-compose (with hyphen), they need updating. The docker-compose binary is no longer maintained. Update all scripts to docker compose (with space).

Q44 Container anti-patterns - fat containers, multiple processes

The Simple Answer

Anti-patterns are common mistakes that work in development but cause problems in production.

Fat containers - Installing every possible tool 'just in case.' Each tool increases image size and attack surface. Install only what the application needs to run.

Multiple processes - Running a web server, background worker, and cron scheduler in one container. This defeats the purpose of containers. If one process crashes, you lose all three. Run each process in its own container.

Storing data in the container filesystem - Data disappears when the container is recreated. Always use volumes for persistent data.

Running as root - Gives attackers maximum privileges if they exploit a vulnerability. Always use a non-root USER.

Using latest tag - Unpredictable. Two deployments might get different images. Always pin versions.

Ignoring health checks - Without them, orchestrators cannot detect broken containers and will keep routing traffic to them.

Q45 Docker daemon configuration - daemon.json, log rotation

The Simple Answer

The Docker daemon is configured through /etc/docker/daemon.json. Key production settings include log rotation, storage driver selection, default runtime options, and registry mirrors.

Critical Production Settings

Log rotation is essential - without it, container logs grow until the disk is full. Set default-address-pools to control the IP ranges Docker uses for container networks. Set storage-driver to overlay2 explicitly. Configure insecure-registries only if absolutely necessary and understand the security implications.

Q46 Docker Swarm basics - when it still makes sense over Kubernetes

The Simple Answer

Docker Swarm is Docker's built-in orchestration. It turns a pool of Docker hosts into a single virtual host. While Kubernetes dominates the orchestration landscape, Swarm still has legitimate use cases.

When Swarm Makes Sense

Small teams with simple deployment needs. Applications with fewer than 20 services. Teams that do not have Kubernetes expertise and cannot justify the learning curve. Edge deployments where Kubernetes is too heavy. Swarm is built into Docker - no additional installation, no additional components.

When to Choose Kubernetes Instead

Complex microservice architectures. Multi-cloud deployments. Advanced scheduling requirements. Strong ecosystem of tools and operators. Industry standard skills and hiring advantages.

Q47 Container design patterns - sidecar, ambassador, adapter, init

The Simple Answer

These patterns describe how multiple containers work together in a pod or service group. They come from Kubernetes but the concepts apply to any container orchestration.

Sidecar - A helper container running alongside the main application. Examples: log shipper, metrics exporter, TLS proxy. The sidecar enhances the main container without modifying it.

Ambassador - A proxy container that handles outbound connections on behalf of the main container. Example: a container that routes database connections to the correct shard.

Adapter - Transforms the main container's output into a standard format. Example: converting application-specific metrics into Prometheus format.

Init - Runs before the main container and performs setup tasks. Example: running database migrations, downloading configuration, waiting for dependent services.

Q48 Docker context and managing remote Docker hosts

The Simple Answer

Docker contexts let you switch between multiple Docker daemons from a single client. Instead of changing the `DOCKER_HOST` environment variable, you create named contexts and switch between them with `docker context` use. Useful for managing development, staging, and production Docker hosts from your local machine.

SECTION 3 - Senior Architecture, Internals & Security (Q49-Q62)

Q49 Container internals - namespaces, cgroups, union filesystems

The Simple Answer

Containers are not a single technology - they are a combination of Linux kernel features working together. Namespaces provide isolation. Cgroups provide resource limits. Union filesystems provide the layered image storage.

Namespaces - Isolation

Each namespace creates an isolated view of a system resource. PID namespace gives each container its own process tree starting from PID 1. Network namespace gives each container its own IP address, routing table, and ports. Mount namespace gives each container its own filesystem view. User namespace maps container UIDs to different host UIDs for rootless containers. IPC and UTS namespaces isolate inter-process communication and hostname respectively.

Cgroups - Resource Control

Control groups limit and account for resource usage. Memory cgroup enforces memory limits and triggers OOM kills. CPU cgroup controls CPU time allocation. I/O cgroup limits disk throughput. Cgroups are what make resource limits in docker run actually work.

Union Filesystems - Layered Storage

Union filesystems like OverlayFS layer multiple directories on top of each other. Lower layers are read-only (image layers). The top layer is read-write (container layer). When a container modifies a file, it is copied from the lower layer to the upper layer (copy-on-write). This is why Docker images can share base layers efficiently.

Understanding these internals is what separates senior from mid-level candidates. When someone says 'a container is just a process with some isolation,' they are essentially correct. There is no container hypervisor, no container kernel. It is the same kernel running the same processes, just with restricted views and resource limits.

Q50 OCI standards - image spec, runtime spec, distribution spec

The Simple Answer

The Open Container Initiative (OCI) defines industry standards for container formats and runtimes. This standardization means images built with Docker work with Podman, containerd, CRI-O, and any OCI-compliant tool.

Image Specification - Defines the format of container images - how layers are structured, how metadata is stored, how manifests reference layers.

Runtime Specification - Defines how to run a container from an unpacked image. Specifies the lifecycle operations: create, start, kill, delete. The reference implementation is runc.

Distribution Specification - Defines how to push and pull images from registries. Standardizes the API that registries expose.

Why this matters in interviews: It demonstrates you understand that Docker is not the only container tool and that the industry has standardized around open specifications. Companies moving to Kubernetes may use containerd or CRI-O instead of Docker, and understanding OCI explains why that works seamlessly.

Q51 Docker vs containerd vs CRI-O vs Podman - the runtime landscape

The Simple Answer

Docker is a complete platform. Underneath it uses containerd as its container runtime. Kubernetes deprecated Docker as a runtime in favor of containerd and CRI-O directly. Podman is a daemonless, rootless Docker-compatible alternative.

Docker - Full platform including CLI, build system, compose, swarm, and the daemon. Uses containerd under the hood.

containerd - Industry-standard container runtime. Manages the complete container lifecycle. Used by Docker, Kubernetes, and cloud providers. Lower-level than Docker - no build system, no compose.

CRI-O - Lightweight container runtime designed specifically for Kubernetes. Implements the Kubernetes Container Runtime Interface (CRI) and nothing else. Minimal attack surface.

Podman - Docker-compatible CLI that runs without a daemon process. Rootless by default. Uses the same commands as Docker. Popular on Red Hat / Fedora systems and as a Docker Desktop alternative.

Q52 Rootless containers - setup, limitations, and security benefits

The Simple Answer

Rootless mode runs the entire Docker daemon and containers without root privileges. Even if an attacker escapes the container, they only have the privileges of a regular user on the host, not root.

Limitations

Cannot use privileged ports (below 1024) without additional configuration. Some storage drivers have limitations. Performance may differ slightly due to user namespace mapping. Network setup requires slirp4netns which is slower than the default bridge networking.

Rootless containers are the strongest single security improvement you can make in a container deployment. The trade-offs are minor compared to the security benefit of removing root from the equation entirely.

Q53 Docker storage drivers - overlay2, how layers mount

The Simple Answer

The storage driver controls how image layers and the container writable layer are stored on disk. OverlayFS (overlay2 driver) is the default and recommended driver for nearly all deployments.

How overlay2 Works

OverlayFS creates two directories - lowerdir (read-only image layers) and upperdir (read-write container layer). A merged view combines both. When the container reads a file, OverlayFS checks the upper layer first, then falls through to lower layers. When the container writes a file that exists in a lower layer, it copies it to the upper layer first (copy-on-write), then modifies the copy.

Q54 Container networking internals - veth pairs, iptables, bridge, DNS

The Simple Answer

Docker's bridge networking creates a virtual ethernet bridge on the host. Each container gets a virtual ethernet pair (veth) - one end in the container's network namespace, the other on the host bridge. Docker configures

iptables rules for port publishing (DNAT) and inter-container communication. An embedded DNS server handles name resolution.

The Traffic Flow

Container to internet: container eth0 → veth pair → docker0 bridge → host iptables NAT → host eth0 → internet. Published port from internet: internet → host eth0 → iptables DNAT → docker0 bridge → veth pair → container eth0.

Q55 Image layer deduplication, squashing, and registry garbage collection

The Simple Answer

Docker images share layers. If ten images all start FROM python:3.12-slim, that base image is stored once. This deduplication saves significant disk space and network bandwidth. Squashing merges all layers into one, which can reduce size but eliminates sharing. Registry garbage collection removes unreferenced layers.

When to Squash

Squashing makes sense when layers contain files that were added and then deleted in subsequent layers - the deleted files still exist in earlier layers. Squashing eliminates these intermediate artifacts. The trade-off is losing layer sharing with other images.

Q56 BuildKit architecture - frontend, LLB, cache export/import

The Simple Answer

BuildKit is Docker's modern build engine, replacing the legacy builder. It introduces parallel build execution, better caching, secret mounts, and the ability to use different build frontends.

LLB (Low-Level Build) - An intermediate representation of the build graph. Like an AST for builds. Enables parallel execution of independent steps.

Frontends - Translate human-readable build definitions (like Dockerfiles) into LLB. The default frontend is the Dockerfile frontend. Alternative frontends exist for other build systems.

Cache Export/Import - BuildKit can export build cache to a registry or local directory and import it on subsequent builds. This is what enables fast CI builds.

Q57 Docker in enterprise - governance, golden images, internal registries

The Simple Answer

Enterprise Docker deployments require governance around which images are allowed, how they are scanned, and who can deploy them.

Golden Images

A golden image is an organization-approved base image that all teams must build from. It includes required security patches, monitoring agents, logging configuration, and compliance tools. Teams build their application layers on top of the golden image.

Governance Pipeline

Developers build from golden base images. CI scans every image for vulnerabilities. Policy engines block images that fail security checks. Only signed images from the internal registry can be deployed to production.

Q58 Image signing and supply chain security - cosign, Notary, SBOM

The Simple Answer

Image signing cryptographically proves that an image was built by a trusted party and has not been tampered with. SBOMs (Software Bill of Materials) document every component inside an image. Together they address supply chain security.

cosign - Part of the Sigstore project. Signs container images using keyless signing with identity-based verification. Integrating with CI to sign every image on build is straightforward.

Notary / Docker Content Trust - Docker's original signing mechanism. More complex to set up than cosign but deeply integrated with Docker.

SBOM - A complete list of every package, library, and dependency in the image. Required by some compliance frameworks. Tools like Syft generate SBOMs from container images.

Q59 Linux capabilities, seccomp profiles, AppArmor for containers

The Simple Answer

Docker drops most Linux capabilities by default, keeping only a minimal set needed for normal operation. You can further restrict containers using custom seccomp profiles and AppArmor policies.

Capabilities

Instead of giving a process full root or no root, Linux capabilities split root privileges into fine-grained permissions. Docker drops capabilities like NET_RAW, SYS_ADMIN, and SYS_PTRACE by default. If your application does not need to bind to privileged ports, drop NET_BIND_SERVICE too. The principle of least privilege - give only what is needed.

Seccomp

Seccomp filters restrict which system calls a container can make. Docker's default profile blocks about 44 dangerous syscalls. Custom profiles can restrict further based on your application's needs.

Q60 Docker Bench for Security - what it checks and how to act on findings

The Simple Answer

Docker Bench for Security is an open-source script that checks dozens of security best practices against your Docker deployment. It covers host configuration, Docker daemon configuration, container runtime security, and image security.

Key Checks

Host configuration: kernel auditing, Docker version, separate partition for /var/lib/docker. Daemon configuration: enabling user namespace support, configuring TLS, limiting container privileges. Images: no unnecessary packages, no secrets, non-root users. Containers: read-only root filesystem, no privileged containers, resource limits set.

Run Docker Bench regularly - at least monthly - and track findings over time. Many organizations run it on first setup and never again. Security posture degrades as configurations drift and new containers are deployed without proper review.

Q61 Docker socket security - risks of mounting /var/run/docker.sock

The Simple Answer

The Docker socket gives full control over the Docker daemon. Mounting it into a container gives that container the ability to create, destroy, or modify any container on the host, access any volume, and effectively gain root access on the host machine.

The Risk

A compromised container with socket access can: start a privileged container with the host filesystem mounted, read any secret in any volume, stop or replace any container, modify the Docker daemon configuration, and effectively own the host.

When It Is Necessary and How to Mitigate

Some tools need socket access - monitoring agents, CI builders, reverse proxies like Traefik. Mitigations: use socket proxies like Tecnativa's docker-socket-proxy that filter allowed API calls, use read-only mount when only inspection is needed, run the container in a restricted network, and monitor socket access with audit logging.

Q62 Migrating from Docker Compose to Kubernetes - what changes and what doesn't

The Simple Answer

The container images themselves do not change. What changes is how you define networking, storage, scaling, configuration, and deployment strategy.

What Maps Directly

Services become Deployments + Services. Named volumes become PersistentVolumeClaims. Environment variables become ConfigMaps and Secrets. Ports become Service ports.

What Changes Fundamentally

Docker Compose's networking is flat - every service can reach every other service by name. Kubernetes requires explicit Service definitions and NetworkPolicies. Docker Compose health checks become Kubernetes liveness and readiness probes. Docker Compose depends_on becomes Kubernetes init containers. Scaling goes from replicas in Compose to HPA in Kubernetes.

Tools like Kompose can convert docker-compose.yml to Kubernetes manifests, but the output is a starting point, not production-ready. Kubernetes has concepts - like rolling update strategy, resource requests vs limits, pod disruption budgets - that have no Docker Compose equivalent and must be configured from scratch.

SECTION 4 - Tricky Scenarios & Real-World Disasters (Q63-Q80)

Q63 Container keeps restarting in a loop - debug step by step

Step-by-Step

- Check docker logs container_name for application error messages
- Check docker inspect container_name for exit code - 137 means OOM, 1 means app error, 139 means segfault
- If OOM killed, increase memory limits or fix the memory leak
- If application error, override the entrypoint to get a shell and investigate: `docker run -it --entrypoint /bin/sh image_name`
- Check if the container depends on a service that is not ready - database, API, config server
- Verify environment variables are set correctly for this environment
- Check if a recent image change introduced the issue by running the previous image version

The most common cause of restart loops is a missing environment variable or an unreachable dependency. Always check configuration and connectivity before diving into application code.

Q64 Docker build taking 20 minutes - optimize to under 2 minutes

Diagnosis

- Check build context size - is .dockerignore missing or incomplete?
- Check instruction ordering - is COPY . happening before dependency installation?
- Check if every build pulls a base image - pin the image and cache it
- Check if each RUN downloads packages - combine into one layer and cache
- Check if multi-stage builds are being used

The Fix Recipe

Add a proper .dockerignore. Reorder to COPY dependency files first, install dependencies, then COPY source code. Use BuildKit with registry caching in CI. Use multi-stage builds to parallelize independent steps. Consider using a build cache image that is pushed and pulled.

Q65 Image size is 2.3GB - reduce without breaking the application

The Systematic Approach

- Run `docker history` to see which layers are largest
 - Switch from full base image to slim variant - often saves 500 MB+
 - Implement multi-stage build - build in one stage, run in another
 - Combine RUN commands and clean up in the same layer - remove apt caches, npm caches, pip caches
 - Remove development dependencies from the final image
 - Use .dockerignore to exclude test files, docs, and build artifacts from the context
-

Target under 200 MB for most web applications. Under 50 MB for Go and compiled language services. Under 10 MB for static binaries on scratch.

Q66 Container runs locally but crashes in production - what is different?

Common Causes

- Different environment variables - database URLs, API keys, feature flags
- Different resource limits - production has memory limits, local does not
- Different networking - local uses bridge, production uses overlay or host
- Different file permissions - local runs as root, production as non-root
- Different base image - local uses latest, production uses a pinned version
- Missing files - volumes or config maps not mounted correctly in production

Debug Approach

Compare docker inspect output between local and production. Diff the environment variables. Check resource limits. Try running locally with the exact same image tag, environment variables, and resource limits as production.

Q67 Secrets leaked in a Docker image layer - detect and remediate

Immediate Response

- Rotate every secret that was in the image immediately - assume they are compromised
- Remove the image from the registry
- Rebuild the image without the secrets using BuildKit secret mounts
- Audit docker pull logs to see who may have pulled the compromised image

Detection

Use tools like TruffleHog, GitLeaks, or Trivy's secret scanning to check images for embedded secrets. Run these scans in CI on every build. For existing images, run `docker save image | tar xf -` and grep through the layers.

Prevention

Never COPY secret files. Never use ARG for secrets. Use BuildKit `--mount=type=secret` for build-time secrets. Use runtime environment variables or mounted secrets for run-time secrets.

Q68 Database container lost all data after restart - why and how to fix

Why It Happened

The database was running without a volume mount. All data was stored in the container's writable layer. When the container was recreated - even from the same image - the writable layer was fresh and empty. The data was gone.

Recovery

If the old container still exists (stopped but not removed), its writable layer is intact. Use `docker cp` to extract the data directory. If the container was removed, the data is gone unless the host filesystem was not cleaned and someone can recover files from `/var/lib/docker`.

Prevention

Always declare volumes for database data directories. Test recovery by deliberately deleting and recreating the container. Implement automated backups to external storage. Document the volume mount requirement in the project's README.

Q69 Two containers cannot communicate on the same host - troubleshoot

Checklist

- Are both containers on the same Docker network? `docker network inspect`
- Are they on the default bridge (no DNS resolution) vs a custom bridge (DNS works)?
- Is the target container healthy and listening on the expected port?
- Is the source container using the correct hostname - container name, not localhost
- Are there iptables rules blocking traffic?
- Is the Docker daemon's ICC (inter-container communication) setting disabled?
- Are there port conflicts or incorrect port mappings?

The most common cause: One container is on the default bridge, the other is on a custom network. They cannot communicate because they are on different networks. Put both containers on the same custom network.

Q70 Docker Hub rate limits hit during CI - workaround strategies

Solutions

- Authenticate all Docker Hub pulls - authenticated users get 200 pulls per 6 hours vs 100 for anonymous
- Set up a pull-through cache registry using Harbor or a registry mirror
- Mirror frequently used images to your private registry (ECR, GCR)
- Use cloud provider base images instead of Docker Hub - AWS `public.ecr.aws`, Google's `gcr.io`
- Cache base images in your CI system between builds
- Reduce the number of pipeline stages that pull images - share images between stages

Q71 Production container compromised - incident response

Immediate Response

- Isolate the container - disconnect from network but do not kill it yet
- Capture a snapshot: `docker commit container_name forensics:${date +%Y%m%d}`
- Capture logs: `docker logs container_name > incident_logs.txt`
- Kill the container and deploy a known-good image
- Rotate all credentials the container had access to

Investigation

Analyze the committed forensics image - check for unexpected processes, modified files, new user accounts, suspicious network connections. Review how the attacker got in - vulnerable application code, unpatched base image, exposed Docker socket, leaked credentials.

Prevention

Image scanning in CI. Rootless containers. Read-only root filesystem. No Docker socket mounting without proxy. Network policies. Regular base image updates.

Q72 Legacy monolith to containerized microservices - migration strategy

Phased Approach

Phase 1 - Containerize the monolith as-is. Get it running in Docker without changing any code. This gives you reproducible builds and deployment consistency. Do not try to split services yet.

Phase 2 - Extract the first service. Pick the least coupled component. Run it as a separate container. The monolith calls it over the network. Verify everything works.

Phase 3 - Iterate. Extract services one at a time. Each extraction is a separate project with its own testing and rollback plan. Move from the edges inward.

The worst strategy is the big bang rewrite. Rewriting everything as microservices at once takes years, costs millions, and often fails. Gradual extraction is slower per step but dramatically safer overall.

Q73 Container OOM killed but no application logs - diagnose without logs

Why There Are No Logs

When the OOM killer strikes, it sends SIGKILL - the process terminates instantly with no chance to flush log buffers. Any buffered log messages are lost. The application did not crash gracefully.

Diagnosis

- `docker inspect` shows exit code 137 and `OOMKilled: true`
- Check `dmesg` or `journalctl` on the host for kernel OOM messages showing which process was killed and how much memory it was using
- Run the container without memory limits and monitor memory growth with `docker stats`
- Use a memory profiler inside the container to identify the leak or peak usage
- Check if the application loads large datasets into memory at startup

Q74 Volume permission denied - uid/gid mismatch between host and container

The Problem

The container runs as non-root user (uid 1000) but the mounted volume directory on the host is owned by root (uid 0). The container process cannot read or write files in the volume. Permission denied.

Solutions

- Match the container user's uid/gid to the host directory owner: `RUN usermod -u 1000 appuser`
- Set volume permissions in an init script that runs as root before switching to the non-root user
- Use named volumes instead of bind mounts - Docker manages permissions
- Use `--user` flag to run as the specific uid that owns the host directory
- In Kubernetes, use `securityContext.fsGroup` to automatically adjust volume permissions

Q75 Zombie processes accumulating inside a container

The Problem

The main application spawns child processes. When children exit, they become zombie processes. Normally PID 1 (init) would reap them. But the application is PID 1 and does not call `wait()` on its children. Zombies accumulate until the process table is full.

Detection

`docker exec container_name ps aux` shows processes in Z state (zombie). `docker top container_name` shows process list.

Fix

Use `docker run --init` to add tini as PID 1. Tini properly reaps zombie processes. Alternatively, install tini in the Dockerfile and use it as the ENTRYPOINT. For Python applications, use signal handlers or the `python-prctl` library to handle child process cleanup.

Q76 /var/lib/docker eating 100% disk space

Common Causes

- Unrotated container logs - json-file driver with no max-size setting
- Dangling images from old builds never cleaned up
- Orphaned volumes from deleted containers
- Build cache accumulated over months of builds
- Large container writable layers from applications writing temp files

Clean Up

Run `docker system df` to see what is consuming space. `docker system prune -a --volumes` removes everything unused. For logs, configure log rotation in `daemon.json` with `max-size` and `max-file`. Set up a cron job to run `docker system prune` periodically on CI runners and build servers.

Prevention is better than cleanup. Configure log rotation in `daemon.json` from day one. Set up image lifecycle policies in your registry. Use CI jobs that clean up after themselves. Monitor `/var/lib/docker` disk usage with alerting.

Q77 DNS resolution failing intermittently inside containers

Common Causes

- Docker's embedded DNS server has race conditions under high load
- The `ndots` setting in `resolv.conf` causes unnecessary DNS search domain lookups, amplifying DNS traffic
- Host DNS configuration changes not reflected inside running containers
- Alpine-based images using musl's DNS resolver which behaves differently from glibc
- Docker network MTU mismatch causing packet fragmentation

Fixes

Set `ndots:1` in DNS config to reduce unnecessary lookups. Use `dnsmasq` or `CoreDNS` as a local DNS cache. Avoid Alpine if DNS issues persist - switch to Debian slim. Check MTU settings match between Docker networks and the host network. Restart the Docker daemon to refresh DNS configuration after host network changes.

Q78 App ignores SIGTERM and takes 30 seconds to kill

Why It Happens

The Dockerfile uses shell form CMD (CMD python app.py). This wraps the command in /bin/sh -c, making the shell PID 1. When Docker sends SIGTERM, the shell receives it but does not forward it to the Python process. After the grace period (default 10 seconds), Docker sends SIGKILL.

Fix

Switch to exec form: CMD ["python", "app.py"]. The Python process becomes PID 1 and receives SIGTERM directly. Then add signal handling in the application code to catch SIGTERM and shut down cleanly - close database connections, finish in-flight requests, flush buffers.

If you need shell features (variable expansion), use exec: CMD ["sh", "-c", "exec python app.py"]. The exec replaces the shell with the Python process, making Python PID 1.

Q79 Compromised base image in production - supply chain attack

Immediate Response

- Identify all images built from the compromised base - check your CI build history
- Rebuild all affected images from a known-good base image version
- Deploy rebuilt images to production immediately
- Audit logs for any suspicious activity during the exposure window
- Rotate all credentials that were accessible to affected containers

Prevention

Pin base images by digest (sha256) not just by tag - tags can be overwritten. Scan every image in CI before pushing. Sign images and verify signatures before deploying. Use an approved golden base image maintained by your security team. Monitor base image sources for security advisories. Consider running a private mirror of base images that goes through a vetting process before becoming available internally.

Q80 Multi-stage build cache miss every CI run

Why It Happens

CI environments start fresh - no local Docker cache. Without remote caching configured, every build starts from scratch. Even with remote caching, certain patterns break cache: changing the base image tag, modifying early layers, COPY commands that include frequently changing files.

Diagnosis

Build with --progress=plain to see which steps are cached and which are rebuilding. Check if the cache-from image exists in the registry. Verify the Dockerfile instruction order - a changed file early in the Dockerfile invalidates all subsequent layers.

Fix

Enable BuildKit with --cache-to and --cache-from pointing to your registry. Ensure dependency files (requirements.txt, package-lock.json) are COPYed before source code. Use inline cache mode for simplicity or registry cache mode for better performance. Verify that the CI runner can actually pull from the cache registry with proper authentication.

Interview Tips - How to Use This Guide

For Junior Roles

Focus on Q1-Q22. Be able to write a Dockerfile from scratch and explain each instruction. Understand images vs containers, volumes vs bind mounts, and the basic networking model. Demonstrate you can debug a crashing container using logs and inspect.

For Mid-Level Roles

Q1-Q48 should be comfortable. Be ready to discuss multi-stage builds, CI/CD caching, security scanning, and the PID 1 problem. Have a story about a production Docker issue you debugged. Know how to optimize an image from 1GB to under 100MB.

For Senior Roles

All 80 questions. The internals questions (namespaces, cgroups, overlay2) separate senior from mid-level. Supply chain security, rootless containers, and the container runtime landscape are differentiators. Be ready to discuss migration strategies and enterprise governance.

For Scenario Questions

Pause before answering. State what you would check first, what tools you would use, and what the most common root cause is. Then walk through the fix and the prevention strategy. Interviewers want to see a systematic debugging methodology, not just the answer.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Image vs Container	Image is the template. Container is the running instance. Containers are ephemeral.
CMD vs ENTRYPOINT	ENTRYPOINT is the fixed command. CMD provides default overridable arguments. Always use exec form.
COPY vs ADD	Always use COPY. ADD has implicit behavior that causes surprises.
Layer caching	Put rarely-changing instructions first. Copy dependency files before source code.
Multi-stage builds	Build in one stage, run in another. Keeps final image small and secure.
Volumes	Always use volumes for persistent data. Named volumes for databases. Bind mounts for development.
Non-root	Always add a USER instruction. Defense in depth against container escapes.
Health checks	Always add HEALTHCHECK. Without it, orchestrators cannot detect broken containers.
PID 1 problem	Use --init or tini. Prevents zombie processes and signal forwarding issues.
Graceful shutdown	Use exec form CMD. Handle SIGTERM in your application. Test that containers stop cleanly.
Secrets	Never use ARG or COPY for secrets. Use BuildKit secret mounts for build-time. Use runtime env vars or mounted secrets for run-time.
Tagging	Never use latest in production. Use git SHA or semver. Pin base images by digest.
Log rotation	Configure max-size and max-file in daemon.json. Without it, logs fill the disk.
Image scanning	Run Trivy or equivalent in CI on every build. Block critical vulnerabilities.
Docker socket	Mounting gives full host control. Use socket proxies. Never mount in untrusted containers.
Base images	Debian slim for compatibility. Alpine for small size. Distroless for security. Scratch for static binaries.

Build caching in CI	Use BuildKit with --cache-to and --cache-from. Without it, every CI build starts fresh.
Container networking	Always use custom bridge networks. Default bridge has no DNS resolution.
Exit codes	0 = clean exit. 1 = app error. 137 = OOM killed. 139 = segfault. 143 = SIGTERM received.
Anti-patterns	No fat containers. No multiple processes. No data in writable layers. No latest tag. No running as root.